

SISTEMA DE GESTÃO FINANCEIRA A3: ARQUITETURA DISTRIBUÍDA, SEGURANÇA E QUALIDADE COM SPRING BOOT E JAVASCRIPT

BORRI, Thiago Luvizotto; CHAGAS, Guilherme Ledo; CORRÊA, Diogo Kranz de Araújo; CARMO, Raul Vinicius Pereira do; GRZYBOWSKI, Kauã Vicente; GIMNY, Iracema Parzewski; LIMA, Jailma Lopes de; LISBOA, Bruno Aurelio de; MORAES, Elias de Carvalho; OLIVEIRA, André Luiz Cavalcante Lino; SANTOS, Reinan Pereira dos; SOARES, Frederico Fernandes; TORNERO, Max Dorta; VOSS, Wellinton Matheus.

9245110522@ulife.com.br; 825129608@ulife.com.br; 12525190374@ulife.com.br; 82510096@ulife.com.br; 942515023@ulife.com.br; 9425111059@ulife.com.br; 12625126234@ulife.com.br; 1352510531@ulife.com.br; 12525131288@ulife.com.br; 1262516323@ulife.com.br; 7225114570@ulife.com.br; 32511210@ulife.com.br; 942519540@ulife.com.br; 942519768@ulife.com.br.

Resumo – Este artigo documenta o sistema de gestão financeira A3AnimaGestaoFinanceira, desenvolvido como avaliação A3 da Unidade Curricular de Sistemas Distribuídos e Mobile. O sistema é uma aplicação web distribuída composta por três camadas: SPA JavaScript, API REST com Spring Boot 3 e banco PostgreSQL na plataforma Supabase. São descritos os fundamentos teóricos das principais decisões arquiteturais, cobrindo o estilo REST (Fielding, 2000), segurança com controle de acesso baseado em papéis (Ferraiolo et al., 2001), persistência relacional com JPA/Hibernate e padrões de estabilidade para sistemas distribuídos (Tanenbaum e Van Steen, 2017). Onze testes de integração verificam os fluxos críticos do sistema com banco em memória.

Palavras-chave – Sistemas distribuídos; REST; Spring Boot; Segurança; Testes de integração.

Abstract – This paper documents the A3AnimaGestaoFinanceira financial management system, developed as an A3 assessment for the Distributed Systems course. The system is a three-tier distributed web application: JavaScript SPA, REST API with Spring Boot 3, and PostgreSQL on Supabase. Theoretical foundations are described covering Fielding's REST style (2000), role-based access control (Ferraiolo et al., 2001), JPA/Hibernate persistence, and distributed systems stability patterns (Tanenbaum and Van Steen, 2017). Eleven integration tests verify critical end-to-end flows using an in-memory H2 database.

Keywords – Distributed systems; REST; Spring Boot; Security; Integration tests.

1. Introdução

Sistemas de informação modernos exigem a integração de múltiplos conceitos: arquitetura de software, segurança, persistência e comunicação distribuída. O

sistema A3AnimaGestaoFinanceira foi desenvolvido para demonstrar essa integração em um produto funcional que atende aos requisitos reais de controle financeiro.

Tanenbaum e Van Steen (2017, p. 2) definem sistema distribuído como “uma coleção de computadores independentes que se apresenta a seus usuários como um único sistema coerente”. Essa definição descreve com precisão o sistema desenvolvido: o usuário interage com uma interface única, sem perceber que suas requisições atravessam a internet até um servidor Java que consulta banco de dados hospedado em nuvem. Toda a comunicação entre o frontend e o backend ocorre via mensagens HTTP, seguindo o estilo arquitetural REST formalizado por Fielding (2000).

O objetivo deste artigo é apresentar as principais decisões de projeto e os fundamentos teóricos que as embasam, evidenciando a aplicação prática dos conceitos estudados em sala de aula.

2. Arquitetura do Sistema

2.1 Camadas e Comunicação

O sistema utiliza uma arquitetura em três camadas: apresentação, aplicação e dados. A camada de apresentação foi implementada como uma SPA (Single Page Application) em JavaScript puro, responsável pela interação com o usuário. A camada de aplicação consiste em uma API REST desenvolvida com Spring Boot 3 e Java 21, responsável pelo processamento das regras de negócio e pela comunicação entre a interface e o banco de dados por meio do protocolo HTTP. Já a camada de dados utiliza o banco PostgreSQL gerenciado pela plataforma Supabase. Segundo Fowler (2002, p. 133), essa estrutura, denominada Layered Architecture, facilita a substituição e manutenção de

componentes sem impactar diretamente as demais camadas.

A comunicação entre cliente e servidor é baseada no estilo REST proposto por Fielding (2000, p. 76), que define seis restrições essenciais: separação entre cliente e servidor, ausência de estado em cada requisição, possibilidade de cache, interface uniforme, arquitetura em camadas e código sob demanda. Cada entidade de negócio possui suas próprias URIs e as operações são realizadas utilizando os verbos HTTP apropriados (GET, POST, PUT, DELETE). O cabeçalho Location retornado nas criações (HTTP 201 Created) está em conformidade com a RFC 7231 (IETF, 2014).

2.2 Padrões de Projeto

O backend segue o padrão MVC organizado em três camadas: Controller, Service e Repository. A camada Controller é responsável por receber as requisições HTTP, validar os dados de entrada, acionar o serviço correspondente e devolver a resposta ao cliente — por exemplo, POST /movimentacoes ou GET /produtos. A camada Service concentra as regras de negócio da aplicação, incluindo validações de domínio, coordenação de operações e chamadas aos repositórios — como verificar disponibilidade de estoque ou calcular o saldo antes de registrar uma movimentação. A camada Repository abstrai o acesso ao banco de dados, centralizando as operações de persistência e desacoplando a lógica de negócio dos detalhes de infraestrutura. Por exemplo, a Controller de Movimentações recebe a requisição, a Service coordena a lógica de domínio, e a Repository persiste o registro no PostgreSQL via Supabase.

2.3 Persistência

O mapeamento objeto-relacional utiliza Hibernate via JPA, que define como os objetos Java são mapeados para tabelas no banco de dados. O Spring Data JPA simplifica o acesso ao banco permitindo criar apenas a interface do repositório — as consultas SQL são derivadas automaticamente a partir dos nomes dos métodos (Walls, 2022, p. 73). As operações de estoque são realizadas com SQL nativo atômico (SET estoque = estoque - :qtd), garantindo integridade mesmo com requisições concorrentes. O pool HikariCP é configurado com keepalive-time de 30 s, connection-timeout de 20 s e max-lifetime de 9 min, parâmetros escolhidos para lidar com a latência da conexão remota ao Supabase.

3. Segurança e Controle de Acesso

O sistema utiliza autenticação baseada em sessão HTTP gerenciada pelo Spring Security. As senhas dos usuários

são armazenadas com BCrypt, algoritmo de hash computacionalmente custoso e amplamente indicado para armazenamento de senhas, pois dificulta ataques de força bruta. O limite de 72 caracteres nas senhas decorre de restrição técnica do próprio BCrypt e é tratado explicitamente nas validações do sistema.

O gerenciamento de permissões adota o modelo RBAC (Role-Based Access Control), formalizado por Ferraiolo, Sandhu e Kuhn (2001) como padrão NIST. O sistema possui três perfis hierárquicos: USER (vendedor), ADMIN e SUPERUSER. As regras são centralizadas na classe SecurityConfig e reforçadas com @PreAuthorize nos métodos críticos, implementando defesa em profundidade — se uma camada falhar, a outra ainda protege o recurso.

O filtro UsuarioSessaoRefreshFilter recarrega os dados do usuário do banco a cada requisição autenticada. Com isso, alterações administrativas como bloqueio de conta ou mudança de perfil entram em vigor imediatamente, sem que o usuário precise fazer login novamente. Isso resolve o problema clássico de dados de autorização desatualizados em cache, discutido por Tanenbaum e Van Steen (2017, p. 382).

4. Sistemas Distribuídos na Prática

Tanenbaum e Van Steen (2017, p. 17) descrevem diferentes formas de transparência em sistemas distribuídos. O sistema desenvolvido aplica principalmente a transparência de acesso e a transparência de localização. A transparência de acesso ocorre porque o frontend consome os serviços da API por meio de requisições HTTP sem conhecer detalhes de implementação do backend. Já a transparência de localização está presente no acesso ao banco PostgreSQL hospedado na AWS por meio da plataforma Supabase, que é utilizado pela aplicação via JDBC da mesma forma que um banco local. Além disso, o uso do HikariCP simplifica o gerenciamento das conexões com o banco de dados, abstraindo detalhes de infraestrutura da camada de aplicação.

O Teorema CAP, enunciado por Brewer (2000) e formalizado por Gilbert e Lynch (2002), afirma que sistemas distribuídos não conseguem garantir consistência, disponibilidade e tolerância a partições ao mesmo tempo. Como o banco está em outra máquina, acessado pela internet, a configuração do HikariCP priorizou disponibilidade: o connection-timeout de 20 s aguarda a conexão em vez de falhar imediatamente, e o keepalive-time de 30 s evita que o Supabase encerre conexões ociosas sem aviso.

O endpoint /api/v1/health é consultado pelo frontend a cada 30 segundos e também pelo workflow do GitHub Actions a cada dois dias, que executa um SELECT 1

simples para manter o banco ativo na plataforma. Tanenbaum e Van Steen (2017, p. 311) descrevem esse tipo de verificação periódica — o heartbeat — como “o modo mais simples de detecção de falhas em sistemas distribuídos”. O painel de infraestrutura em `/api/v1/infra/stack` complementa essa observabilidade ao exibir versões das dependências, mapa de tabelas do banco e logs SQL em tempo real, sem necessidade de ferramentas externas.

5. Frontend – SPA e Interface

O frontend é uma SPA (Single Page Application) em JavaScript puro, sem frameworks externos. Todas as páginas existem no DOM e são exibidas ou ocultadas via CSS; a navegação não recarrega a página. A comunicação com a API utiliza `fetch` com `async/await`, com tratamento centralizado de erros e `logout` automático apenas em respostas 401.

O controle de acesso é aplicado no frontend ocultando elementos de interface não autorizados para o perfil logado. Essa é uma camada de usabilidade, não de segurança — a proteção real reside no backend. Gráficos interativos com `Chart.js` exibem evolução financeira e proporção de vendas/despesas, com funcionalidade de drill-down ao clicar em pontos de dados. O sistema suporta personalização white label: nome, logo, fundo e rótulos de menus via API, carregados antes mesmo da tela de login.

Paralelamente, houve uma equipe dedicada ao aprimoramento do Design da aplicação, que trabalhou em sincronia com o Back End para a integração das APIs e do banco de dados Supabase, e para a implementação de soluções que mitigassem erros e promovessem a estabilidade do sistema.

6. Qualidade e Testes

Os endpoints foram pensados para a integração da API, servindo como a principal forma de comunicação do servidor com a internet. Atualmente, a aplicação está hospedada em um servidor Oracle no modelo `always free`.

A garantia da qualidade do software foi viabilizada por uma equipe dedicada ao ciclo de testes. Inicialmente, a aplicação foi submetida a um ambiente produtivo na infraestrutura Oracle, mantendo a integridade da versão estável mais recente via repositório GitHub. O papel desempenhado pelo colaborador Frederico Fernandes Soares como analista de QA foi essencial para o projeto, atuando na validação sistemática de cada versão publicada, identificando inconsistências e elaborando relatórios técnicos que nortearam o refinamento contínuo do processo de desenvolvimento.

A arquitetura de testes compreende três classes estruturadas com `JUnit 5` e a anotação `@SpringBootTest`. O conjunto inclui um *smoke test* para

validação do contexto da aplicação, a classe `ApiIntegrationTest`, com cinco cenários que abrangem as operações de CRUD, e a classe `RelatorioIntegrationTest`, com cinco testes focados na lógica financeira e cenários de borda. Conforme preconizado por Tanenbaum e Van Steen (2017), os testes simulam o comportamento real do *frontend*, realizando a autenticação via protocolo HTTP e gerenciando cookies de sessão para as requisições subsequentes.

O isolamento do ambiente de testes é garantido pelo arquivo `application-test.properties`, que instancia um banco de dados H2 *in-memory* com a política `ddl-auto=create-drop`. Seguindo as melhores práticas de desenvolvimento financeiro (Walls, 2022), a precisão monetária é assegurada pelo uso de `BigDecimal` em detrimento de tipos de ponto flutuante. A validação de integridade ocorre em profundidade: na camada de apresentação, no DTO via `Bean Validation` (`@NotBlank`, `@DecimalMin`) e na regra de negócio do *setter* `setEstoque()`, que impede estados de estoque negativos, assegurando a consistência dos dados.

7. Resultados

O sistema resultante é uma aplicação web distribuída funcional com os seguintes fluxos verificados pelos testes de integração: criação de produtos com HTTP 201 e `header Location`; registro de movimentações financeiras parceladas; cálculo correto de saldo, estoque e margem bruta; relatório financeiro com médias diárias e breakdown por categoria; e comportamento correto em cenários de borda (banco vazio, período sem movimentações, parâmetros ausentes retornando HTTP 400).

A Tabela 1 resume os principais componentes do sistema, as tecnologias utilizadas e os respectivos fundamentos teóricos.

Tabela 1. Componentes, tecnologias e fundamentos teóricos.

Componente	Tecnologia	Fundamento Teórico
API REST	Spring Boot 3	Fielding (2000); RFC 7231 (IETF, 2014)
Persistência	JPA/Hibernate + HikariCP	Fowler (2002); Walls (2022)
Segurança	Spring Security + BCrypt	Ferraiolo et al. (2001)
Distribuição	HTTP + Supabase	Tanenbaum e Van Steen (2017)
Erros	RFC 9457	IETF (2023)
Testes	JUnit 5 + H2	Gilbert e Lynch (2002)

8. Discussão

O desenvolvimento mostrou que é viável construir um sistema distribuído completo com tecnologias de código aberto. A escolha por sessão HTTP em vez de JWT simplificou a implementação para o contexto de instância única, mas limita a escalabilidade horizontal caso o sistema precise crescer, como discutido por Tanenbaum e Van Steen (2017, p. 85). Para versões futuras, identificamos a necessidade de uma ferramenta dedicada de migração de banco de dados, como Flyway ou Liquibase, para gerenciar evoluções de schema de forma mais controlada.

Adicionalmente, o projeto foi concebido para demonstrar a viabilidade de arquiteturas *white label* pautadas pela simplicidade técnica. Tal perspectiva ressalta o potencial mercadológico do sistema, facultando à equipe a oferta de suporte especializado até a implantação definitiva em ambiente produtivo, independentemente do nível de maturidade organizacional da entidade contratante.

O desenvolvimento do sistema fundamentou-se em um esforço colaborativo contínuo, estruturado em alinhamentos estratégicos semanais. A adoção do modelo ágil, conforme preconizado por Sommerville (2019), permitiu a adaptação iterativa do produto às expectativas do grupo, assegurando que todos os membros detivessem participação ativa na consolidação das metas e dos resultados finais alcançados.

O papel desempenhado pelo colaborador Guilherme Ledo Chagas foi essencial para o projeto, atuando como *Product Manager* ao gerenciar o repositório GitHub e, em conjunto com Bruno Aurelio de Lisboa, coordenar as etapas evolutivas. Sua atuação consistiu na validação sistemática de componentes via *branches*, norteando o refinamento contínuo das funcionalidades integradas ao projeto.

9. Conclusão

O sistema A3AnimaGestaoFinanceira integra de forma coesa os conceitos essenciais de Sistemas Distribuídos e Mobile: arquitetura REST (Fielding, 2000), controle de acesso RBAC (Ferraiolo et al., 2001), persistência relacional com JPA/Hibernate (Walls, 2022), padrões de estabilidade distribuída (Tanenbaum e Van Steen, 2017; Brewer, 2000) e testes de integração ponta a ponta. O resultado é uma aplicação funcional, segura, testada e observável, com onze testes de integração que verificam os fluxos críticos. Trabalhos futuros incluem adoção de Flyway para migrações de banco e autenticação JWT para viabilizar escalabilidade horizontal.

Agradecimentos

Os autores agradecem aos professores Priscila Borges De Moraes e Jardel Lucas Garcia pela orientação e pelos

critérios de avaliação que motivaram o aprofundamento teórico deste trabalho.

Referências

- Brewer E. Towards Robust Distributed Systems. PODC Keynote; 2000.
- Ferraiolo DF, Sandhu R, Kuhn DR. Proposed NIST standard for role-based access control. ACM Trans Inf Syst Secur. 2001; 4(3):224-274.
- Fielding RT. Architectural Styles and the Design of Network-based Software Architectures [dissertação]. Irvine: UC Irvine; 2000.
- Fowler M. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley; 2002.
- Gilbert S, Lynch N. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. ACM SIGACT News. 2002; 33(2):51-59.
- IETF. RFC 7231 — Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content. 2014.
- IETF. RFC 9457 — Problem Details for HTTP APIs. 2023.
- Spring Framework Documentation. Spring Security Reference. <https://docs.spring.io/spring-security/>
- Supabase Documentation. Architecture Overview. <https://supabase.com/docs>
- Tanenbaum AS, Van Steen M. Sistemas Distribuídos: Princípios e Paradigmas. 3a ed. Pearson; 2017.
- Walls C. Spring in Action. 6th ed. Shelter Island: Manning; 2022.
- Sommerville I. Engenharia de Software. 10a ed. São Paulo: Pearson; 2019.